TECHNICAL REFERENCE · SYSTEM ARCHITECTURE

eBPF-SOC

A kernel-level threat detection *and* graduated enforcement platform — from a single-binary host sensor to a multi-tenant, MSSP-grade security operations control plane.

DOCUMENT
System & Enforcement Architecture

AUDIENCE
Engineering · Security · Architecture

SCALE
~25k LOC Go · ~19k LOC TypeScript

KERNEL FLOOR
Linux ≥ 5.15 · cgroup v2 · eBPF CO-RE

SOURCES
Code + /docs (31 documents)

STATUS
Phase 0–1 implemented; roadmap noted

INTERNAL · TECHNICAL

Contents

1 Introduction & system overview

eBPF-SOC is a proactive, kernel-level threat detection *and enforcement* platform. It observes syscall- and kprobe-level behaviour from inside the Linux kernel using eBPF, correlates chains of activity per process, and — the property that separates it from a passive SOC dashboard — converts risk into **graduated, reversible, cryptographically-audited containment**: throttle → tarpit → quarantine → sever.

2	5	12	5
ENFORCEMENT DATA PLANES (PROCESS + DEVICE)	GRADUATED CONTAINMENT RUNGS	EBPF PROGRAMS IN-KERNEL	GRPC CONTROL-PLANE SERVICES

1.1 What makes it different

Most security tooling stops at *detect-and-alert*, or offers only binary *allow/kill*. eBPF-SOC's unique value is the pair of **Choke Gateways**:

- **Process Choke Gateway** — a per-process state machine keyed on Tetragon's stable `exec_id`. Risk score drives a monotonic climb up a five-rung ladder, each rung a stronger kernel action (cgroup CPU/PID caps → cgroup freeze → SIGKILL).
- **Network / Device Choke Gateway** — the same graduated model applied per **MAC address** on an inline transparent bridge, using hand-written `tc` eBPF to shape or cut a LAN device's *forwarded* traffic (an IoT camera, a laptop) that the host's own cgroup hooks never see.

Both are **reversible where it matters** (throttle/tarpit/quarantine and — for devices — even sever), and every decision is written to a **hash-chained, tamper-evident audit log** that a compliance or cyber-insurance reviewer can independently verify.

1.2 System context

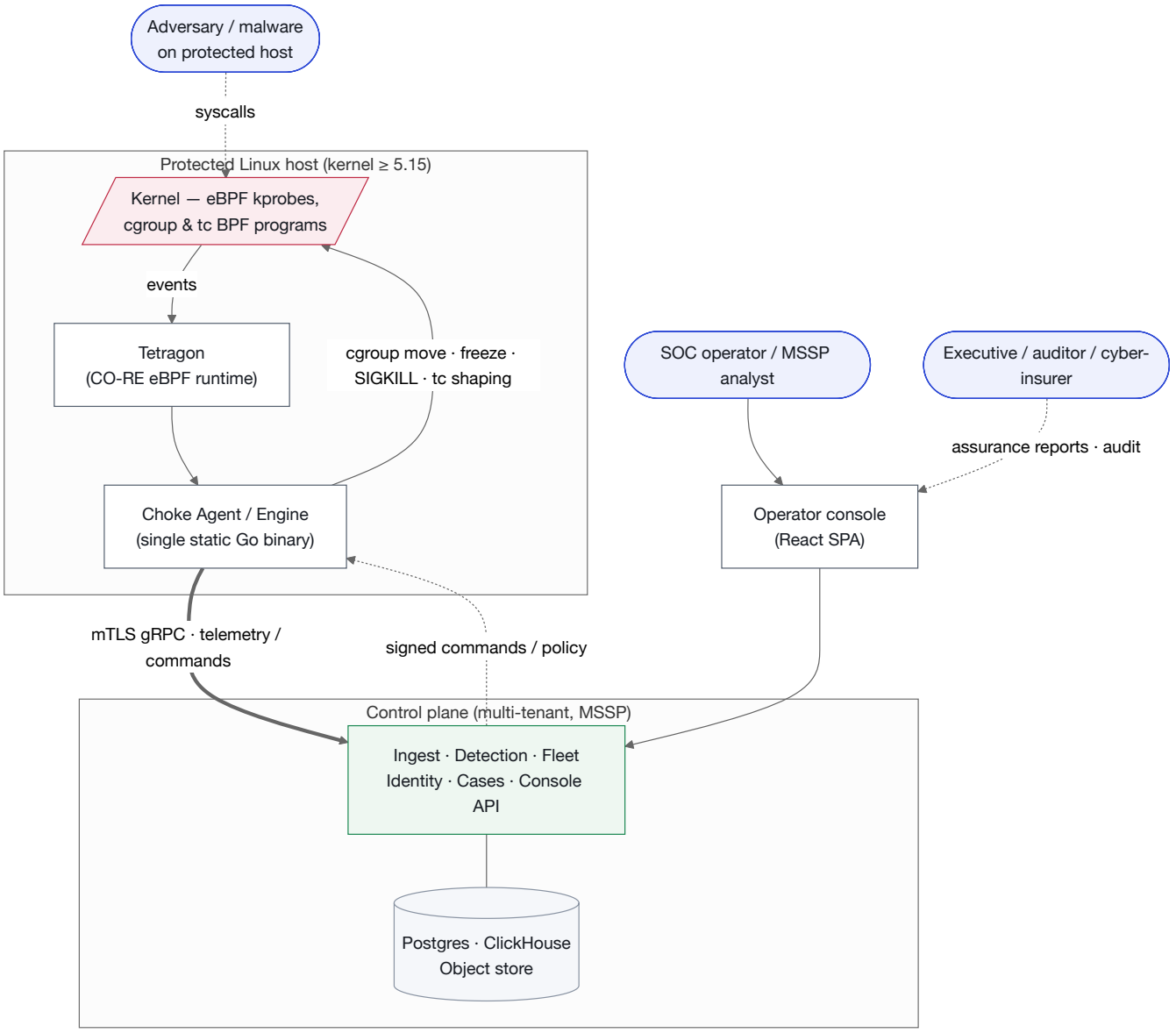


Figure 1.1 — System context. The agent enforces locally and autonomously; the control plane coordinates a fleet across tenants.

1.3 Two deployment shapes, one core

The same `internal/` sensing-and-enforcing core powers two shapes:

SHAPE	BINARY	BEST FOR	STORAGE
Single-host	`cmd/engine`	One box; self-contained sensor + console; air-gapped labs	SQLite (WAL)
Fleet agent	`cmd/agent`	Many hosts reporting to a shared control plane	SQLite WAL as offline buffer → uplink
Control plane	`cmd/controlplane`	Multi-tenant MSSP / enterprise SOC coordination	Postgres + ClickHouse + object store

Two further binaries support development and operations: `cmd/simagent` (synthetic telemetry generator for UI/load work) and `cmd/socbackup` (backup tooling).

> **DESIGN NORTH STAR**
>
> **Autonomous by default, coordinated when connected.** Kernel containment must never depend on the cloud being reachable. If the control plane is down, the agent keeps enforcing its last-applied signed policy and buffers evidence locally until it can drain upstream.

2 Architectural evolution

The platform began as a single root process on one host and is being cut, along one clean seam, into an autonomous per-host **agent** and a horizontally-scalable multi-tenant **control plane** — without regressing any of the properties that make it valuable.

2.1 The seam

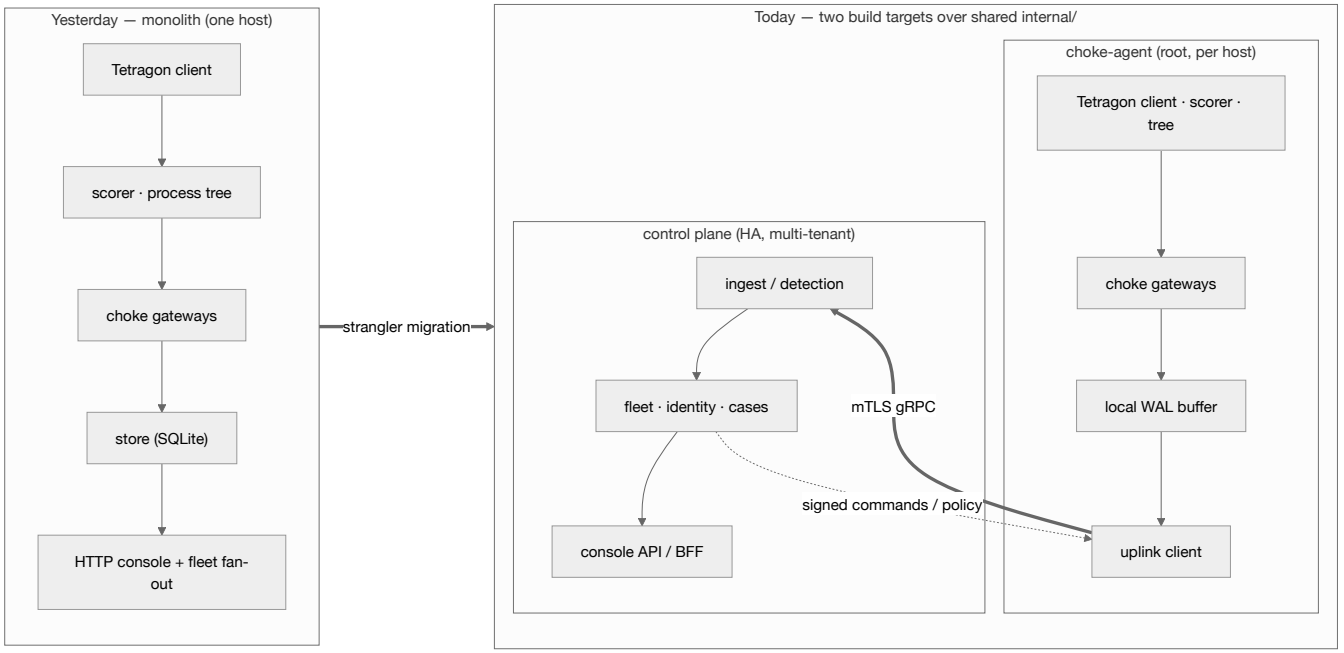


Figure 2.1 — Strangler migration. New `cmd/agent` and `cmd/controlplane` entrypoints wrap the same `internal/` packages; `cmd/engine` keeps building throughout.

2.2 What moved where

PACKAGE(S)	DESTINATION	ROLE AFTER THE SPLIT
`choke` · `enforce` · `score` · `tree` · `device` · `origin` · `sysproc`	Agent	The sensing + enforcing core — ships largely intact.
`store` (SQLite path)	Agent + control plane	Agent: durable offline buffer. CP: central mirror on Postgres/ClickHouse.
`uplink` · `cpclient` · `policypull` · `mtls` · `signing`	Agent	The client half of the wire contract (§10).
`ingest` · `controlplane` · `fleet` · `command` · `enrollment` · `heartbeat`	Control plane	The server half — fan-in, registry, command dispatch, enrollment.
`identity` · `authz` · `bff`	Control plane	OIDC/Keycloak identity, RBAC, and the console backend-for-frontend.
`centralstore` · `bus`	Control plane	Postgres + ClickHouse dialects; NATS event bus.

SACRED INVARIANTS — DO NOT REGRESS

In-kernel graduated enforcement keyed on `exec_id`; offline autonomy; the hash-chained verifiable audit; the device (MAC) choke as a second enforcer class; and single-binary agent ergonomics (statically linked, dependency-free). Only the *console/data* tier is allowed to get heavier.

3 Kernel telemetry layer

Two categories of eBPF program run in the kernel: **Tetragon-generated** programs (from declarative TracingPolicy YAML) that *detect* and feed the scorer, and **hand-written** eBPF C that forms the two *enforcement* data planes. Twelve programs in total.

3.1 Tetragon — the detection sensor

Tetragon (pinned to `quay.io/cilium/tetragon:v1.6.1`) runs as a `--privileged --pid=host` container. It loads each TracingPolicy as one or more CO-RE eBPF programs and streams structured `process_exec` / `process_kprobe` events to the agent over a Unix-socket gRPC stream. `process_exec` fires for every `execve` — the free spine of the process tree.

Detection policies (Post-only → feed the scorer)

POLICY	HOOK	CATCHES	MITRE
`outbound-connections`	kprobe `tcp_connect`	Outbound TCP from shells / netcat family (`bash sh zsh dash nc ncat socat`) — reverse-shell & C2 beacon	T1071
`privilege-escalation`	kprobe `sys_setuid/setreuid`	`setuid(0)` — escalation to root, filtered in-kernel to uid 0	T1548
`sensitive-file-access`	kretprobe `security_file_permission` (LSM)	Reads/writes of `/etc/shadow`, `passwd`, `sudoers`, `/root/.ssh/`, and the honeypot dir	T1003

Enforcement policies (in-kernel Sigkill — independent of engine choke mode)

POLICY	ACTION
`sever-pipe-to-shell`	kprobe `execve` + `Sigkill` on `curl | sh` / `wget | bash` argv postfix — a kernel backstop that fires even if the userspace engine is down.
`override-credential-read`	kretprobe `security_file_permission` + `Sigkill` on credential reads by non-allowlisted binaries, before the read returns data.

> **HONEYPOT SIGNAL**
> The engine seeds five decoy credential files (`_passwd _shadow _id_rsa _aws_credentials _db_backup.sql`) under a watched prefix on startup. No legitimate process ever reads them, so **any hit is a high-confidence signal**, surfaced with a 🍯 badge.

3.2 The two hand-written enforcement data planes

Tetragon can detect and `Sigkill`, but cannot do per-PID token-bucket rate limiting, nor MAC-keyed shaping of *forwarded* traffic. Those two planes are custom eBPF C compiled with `clang -target bpf` and loaded by the pure-Go `cilium/ebpf` library (no libbpf at runtime — the binary stays static).

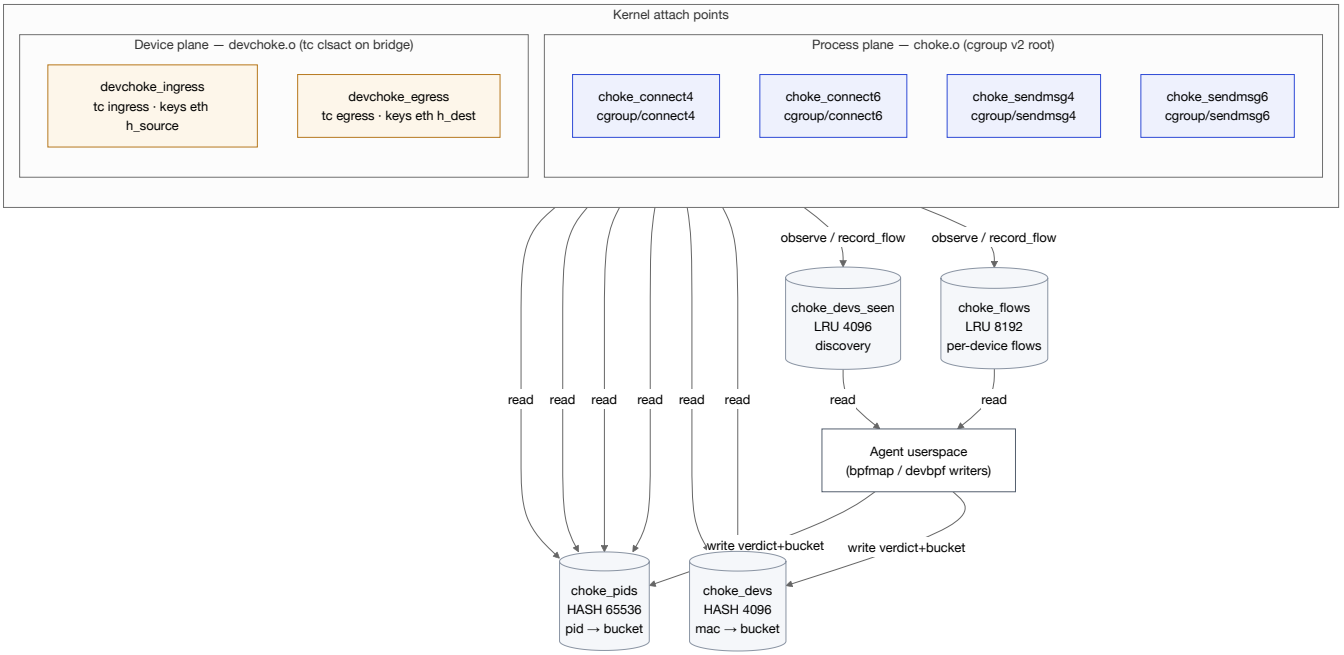


Figure 3.1 — The two enforcement data planes and their BPF maps. Userspace writes verdicts; kernel programs read them on the hot path.

Process plane (`choke.o` , 4 programs)

All four attach at the cgroup-v2 root and gate socket egress for the calling PID by reading `choke_pids` : `SEVER` / `QUARANTINE` deny outright; `THROTTLE` / `TARPIT` run a lazy-refill token bucket and deny when empty. `connect4/6` cover connected TCP/UDP; `sendmsg4/6` close the *connectionless* UDP gap (DNS tunnelling) that never calls `connect()` . This is what makes throttling *actually rate-limit exfiltration* rather than merely log it.

Device plane (`devchoke.o` , 2 programs)

Attached to `tc` clsact on an inline bridge interface. `devchoke_ingress` keys on `eth->h_source` (the device MAC) to shape LAN→WAN; `devchoke_egress` keys on `eth->h_dest` for WAN→LAN. Attaching *both* directions is what lets `sever` cut a device off entirely rather than only choking uploads. Quarantine deliberately still passes DHCP/DNS (`is_infra()`) so a device can re-lease and recover — the very distinction between quarantine and sever. The ingress program also runs passive discovery (`observe()`) and per-device flow recording (`record_flow()`).

> **BYTE-FOR-BYTE MAP LAYOUT**
>
> The 24-byte bucket struct is identical across `choke.c` , `devchoke.c` and the Go structs (`bpfmap.PIDBucket` / `devbpf.DeviceBucket`); `last_ns` is declared first so u64 alignment introduces no padding Go's `encoding/binary` would not emit.

4 The correlation engine

The agent consumes the Tetragon stream, maintains an in-memory process tree keyed by `exec_id`, scores each event, and sums scores along the ancestor walk to a **chain score** — the number that drives both alerting and enforcement.

4.1 Process tree & chain scoring

The tree (`internal/tree`) is keyed on Tetragon's `exec_id`, which survives PID reuse, and is TTL-bounded (~10 min). Per-event scores accumulate down the tree; the chain score is the sum over the ancestor walk (≤ 10 hops). Calibration ensures any single event is at most "low" — "high"/"critical" requires a *chain*, which is what makes detection proactive rather than signature-reactive.

PER-EVENT SCORE CONTRIBUTIONS

`curl|sh` / `wget|sh`	+25
`nc -e` / shell-arg netcat	+20
credential file (shadow, .ssh)	+20
`base64 -d` in args	+15
`setuid(0)` kprobe	+15
outbound TCP from a shell	+12
sensitive file (other)	+8
`chmod +x`	+5
network tool exec	+5

ALERT SEVERITY THRESHOLDS

low	chain ≥ 5
medium	chain ≥ 10
high	chain ≥ 20
critical	chain ≥ 40

Alert severity is *separate* from choke thresholds (§5). An alert is raised at chain ≥ 10; enforcement transitions can begin earlier or later depending on the deployment's choke thresholds.

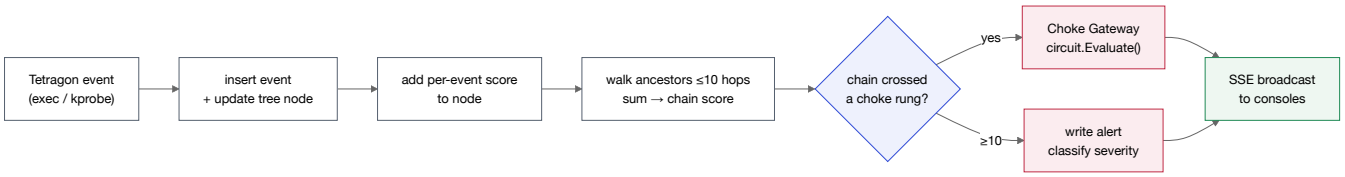


Figure 4.1 — Every event is scored and dispatched to the gateway, not only alert-worthy ones — a process can be throttled before it ever raises an alert.

5 The Choke Gateway — graduated process enforcement

The platform's flagship capability. A monotonic five-rung state machine turns a chain score into a graded, reversible-where-it-matters kernel action, one process (`exec_id`) at a time — a "speed bump → roadblock → freeze → kill" gradient rather than binary block/allow.

5.1 The five-rung ladder

RUNG	DEFAULT THRESHOLD	KERNEL EFFECT	REVERSIBLE	PURPOSE
pristine	< 20	nothing	—	Normal behaviour, no choke
throttled	≥ 20	`choke-throttled` cgroup: 5% CPU, 200 max pids, low IO weight + token-bucket egress	yes	Gentle nudge — capped bandwidth to do harm
tarpit	≥ 50	`choke-tarpit` cgroup: 1% CPU, 50 max pids, lowest IO	yes	Severely degraded — let the attacker reveal their playbook
quarantined	≥ 120	`choke-quarantined` cgroup + `cgroup.freeze = 1`	yes (Thaw)	Frozen mid-syscall — forensics-friendly, state preserved
severed	≥ 200	`kill(pid, SIGKILL)` + bpfmap entry cleared	**no**	Terminal — process gone, PID-reuse safe

The *binary* defaults are 5/15/25/40; the deploy path raises them to **20 / 50 / 120 / 200** so Ubuntu's sshd MOTD churn (which scores ~84) reaches only tarpit — never quarantine (which would freeze sshd and lock the operator out) or sever.

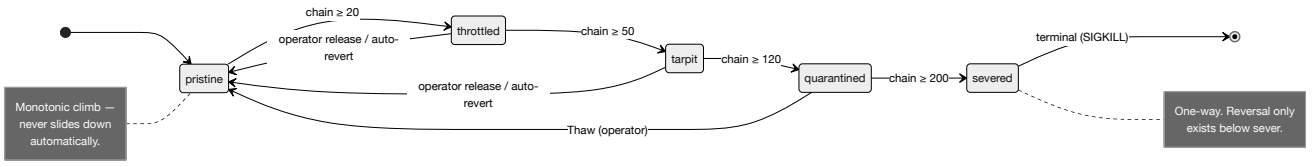


Figure 5.1 — The process choke state machine. Climbs are score-driven and automatic; descents require an audited operator action or a pre-scheduled auto-revert.

5.2 Two design principles

Monotonic. Once a process reaches a rung it never slides back automatically — a transient false-positive spike is not silently undone; the audit trail records "this was suspicious, here is what we did, here is what an operator changed." The only ways down are: an *operator override* (records actor + mandatory reason), a pre-scheduled *auto-revert* ("tarpit for 5 minutes"), a *forget* (drops live state, keeps history), or a *Thaw* (clears the freeze on the whole quarantine tier).

Graduated, not binary. Each tier matches the *cost of being wrong*: throttle is reversible and cheap (a false positive recovers in seconds); sever is unrecoverable and used only when the chain score is unambiguous.

5.3 Composition — circuit, enforcers, store, broadcast

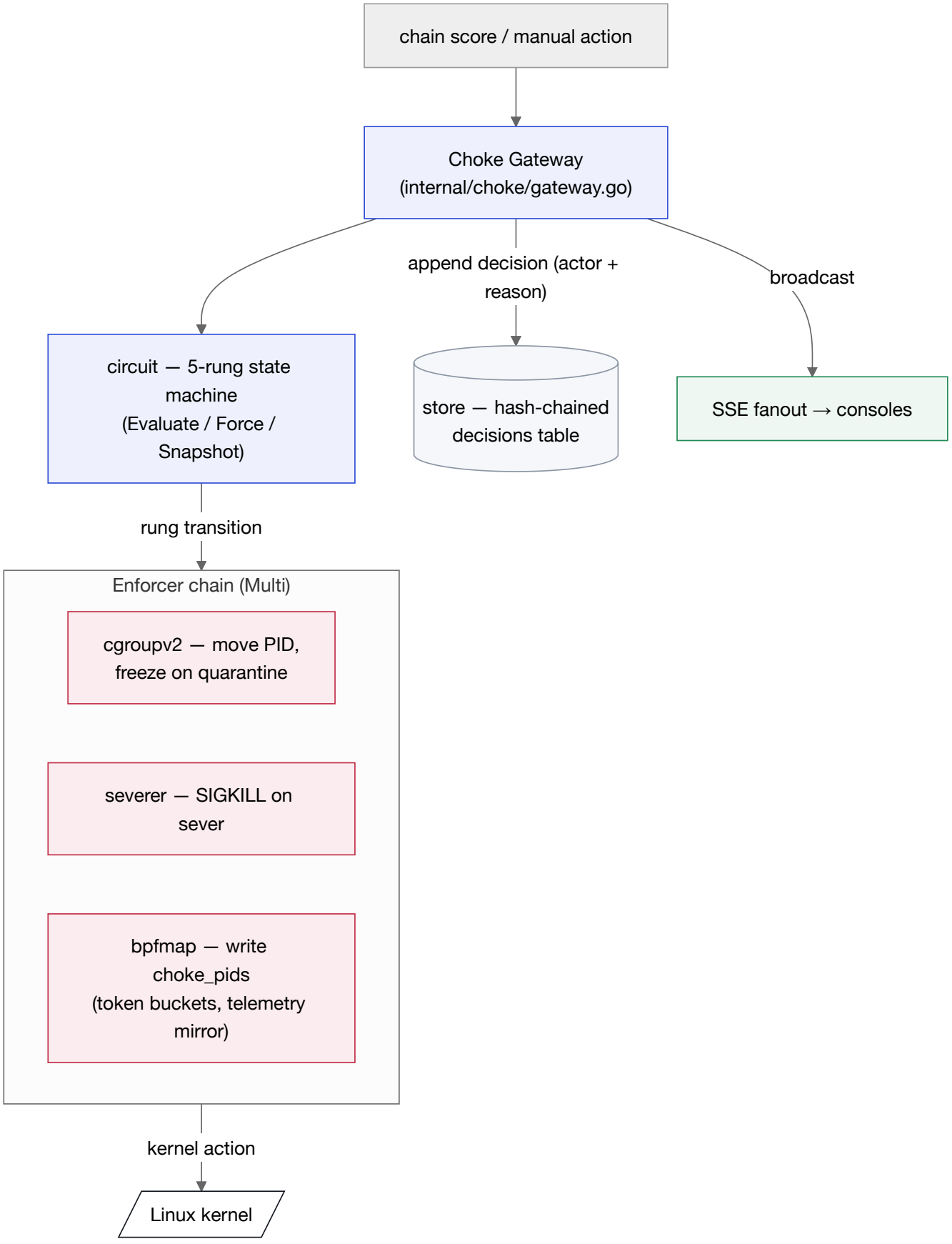


Figure 5.2 — Gateway composition. The circuit decides the rung; the `Multi` enforcer chain actuates it; every transition is audited and broadcast.

Score-driven transitions enter through `gateway.OnEvent()`; operator actions enter through `gateway.Manual()`. Both append a hash-chained row to `decisions` (§7). The enforcer chain is composed via `Multi`: `cgroupv2` physically moves the PID into `/sys/fs/cgroup/choke-<tier>/cgroup.procs` (and sets `cgroup.freeze=1` for quarantine), `severer` issues the SIGKILL for the top rung, and `bpfmap` writes the per-PID token bucket into `choke_pids`. From kprobe firing to PID moved into the matching cgroup is typically sub-millisecond.

6 The Network Choke Gateway — device enforcement

A second, parallel data plane extends the same graduated model from *per-process* to *per-device*. On an inline transparent bridge it throttles or blocks a LAN device's forwarded traffic keyed by MAC address — reaching an IoT camera or laptop that has no agent and that the host's own cgroup hooks can never see.

6.1 Inline-bridge topology

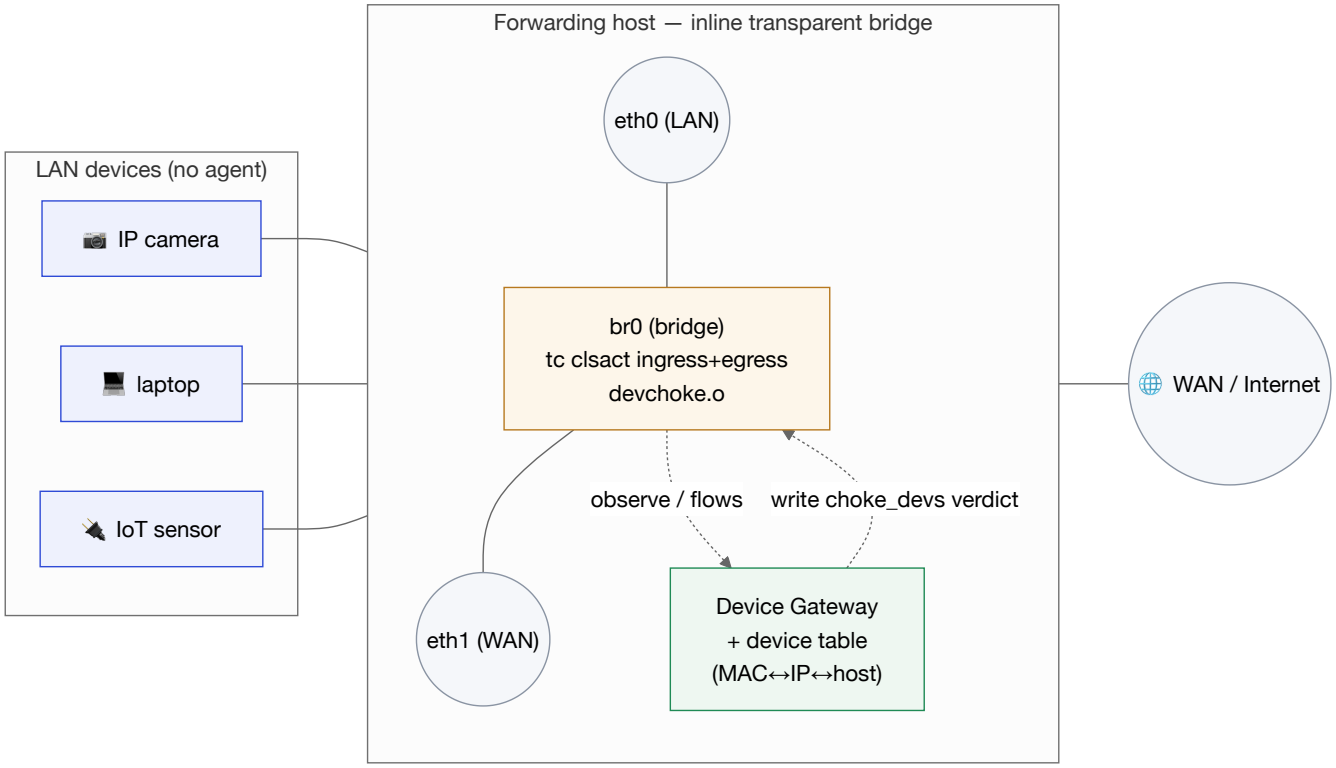


Figure 6.1 — The device choke sits inline on a 2-NIC bridge. All LAN↔WAN frames traverse the tc-attached `devchoke.o`, keyed on device MAC.

6.2 Discovery, flows, and the operator loop

Because a forwarding node has no per-device Tetragon telemetry, there is (today) no automatic score path for the MAC gateway — it is **operator-driven jail/thaw with a full audit trail**. To make that decision informed, the ingress program passively discovers devices (`choke_devs_seen` — last-seen, packet count, sampled source IPv4) and records per-device destination flows (`choke_flows` — MAC, dest addr, dport, proto → packets/bytes). The console's "what is this device talking to?" view reads those maps before an operator chokes.

The device gateway reuses the *same* five-rung circuit as the process gateway. The crucial difference at the top rung: a device `sever` is a **reversible drop rule** — sever cuts the device off, and `thaw` restores it — whereas a process sever is a terminal SIGKILL. Protected MACs (infrastructure, the operator's own management device) refuse quarantine and sever.

INDEPENDENT POSTURES

The two chokes have independent enforcement postures. The process choke follows `--enforce` (flippable at runtime via `/api/choke/mode`); the device choke enforces whenever it has the tc backend and is neither dry-run nor kill-switched. "Observe processes, enforce on devices" is a valid, common posture.

7 The tamper-evident audit chain

Every enforcement decision — automatic or operator-driven, on either gateway — is appended to a hash-chained `decisions` table. Each row's hash incorporates the previous row's hash, so silent deletion or modification of any historical decision breaks verification. This is the compliance and cyber-insurance anchor.

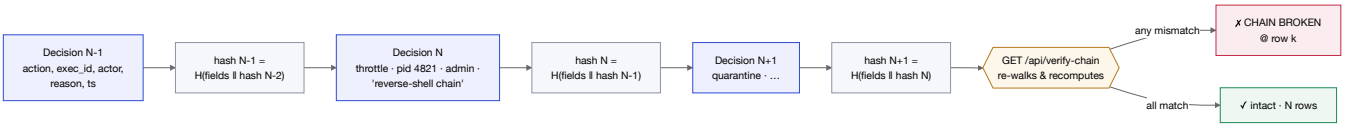


Figure 7.1 — Hash-chained decisions. Deleting or editing any row invalidates every subsequent hash; `/api/verify-chain` pinpoints the first break.

Each decision row records the **actor** (which operator or "system"), a **mandatory reason** for destructive actions (quarantine/sever), the target `exec_id` or MAC, the from/to rung, timestamp, and dry-run flag. In the multi-tenant deployment the chain is mirrored centrally *per tenant*, with the `seq` / `prev_hash` / `hash` fields carried verbatim over the wire so the control plane re-verifies the chain without trusting transport order (§10).

> **ASSURANCE SURFACE**
>
> The console's Assurance lens turns this into board-ready evidence: an integrity badge (intact / broken), the record count, the head hash, and a one-click printable report + JSON "evidence bundle" anchored to the head hash.

8 Data & storage architecture

Storage tiers by role: a pure-Go embedded store at the edge (keeps the agent a single static binary), a relational system-of-record and a columnar analytics store centrally, and object storage for cold/forensic data.

Figure 8.1 — Storage tiers. The `store` dialect seam lets the same schema target SQLite (edge) or Postgres/ClickHouse (central).

8.1 Control-plane relational model (Postgres)

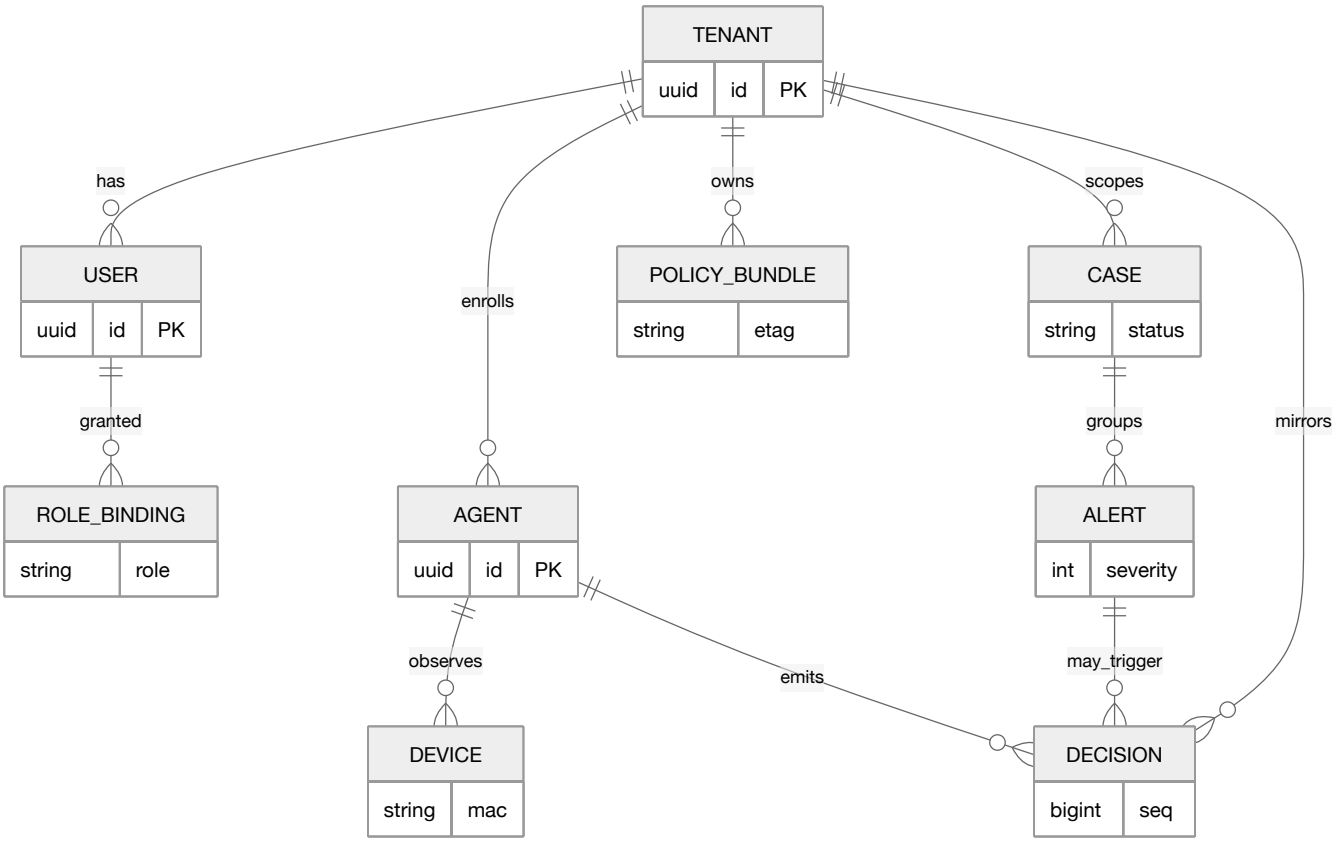


Figure 8.2 — Simplified control-plane entity model. Every row is keyed by `tenant_id`; the shared data-access layer tenant-filters all queries.

The `store` package's `dialect` seam splits cleanly: the agent uses the SQLite path (pure-Go `modernc.org/sqlite`, cgo-free) as an *offline buffer* — events/alerts/decisions are written locally, drained upstream, and survive restarts and partitions; the control plane uses the Postgres dialect (`pgx/v5`) for the relational system-of-record and a ClickHouse dialect (`clickhouse-go/v2`) for firehose-scale event analytics.

9 The multi-tenant control plane

A set of independently-scalable, tenant-aware services behind an API gateway. Each is stateless where it can be; all derive tenant scope from verified identity, never from a client-supplied field.

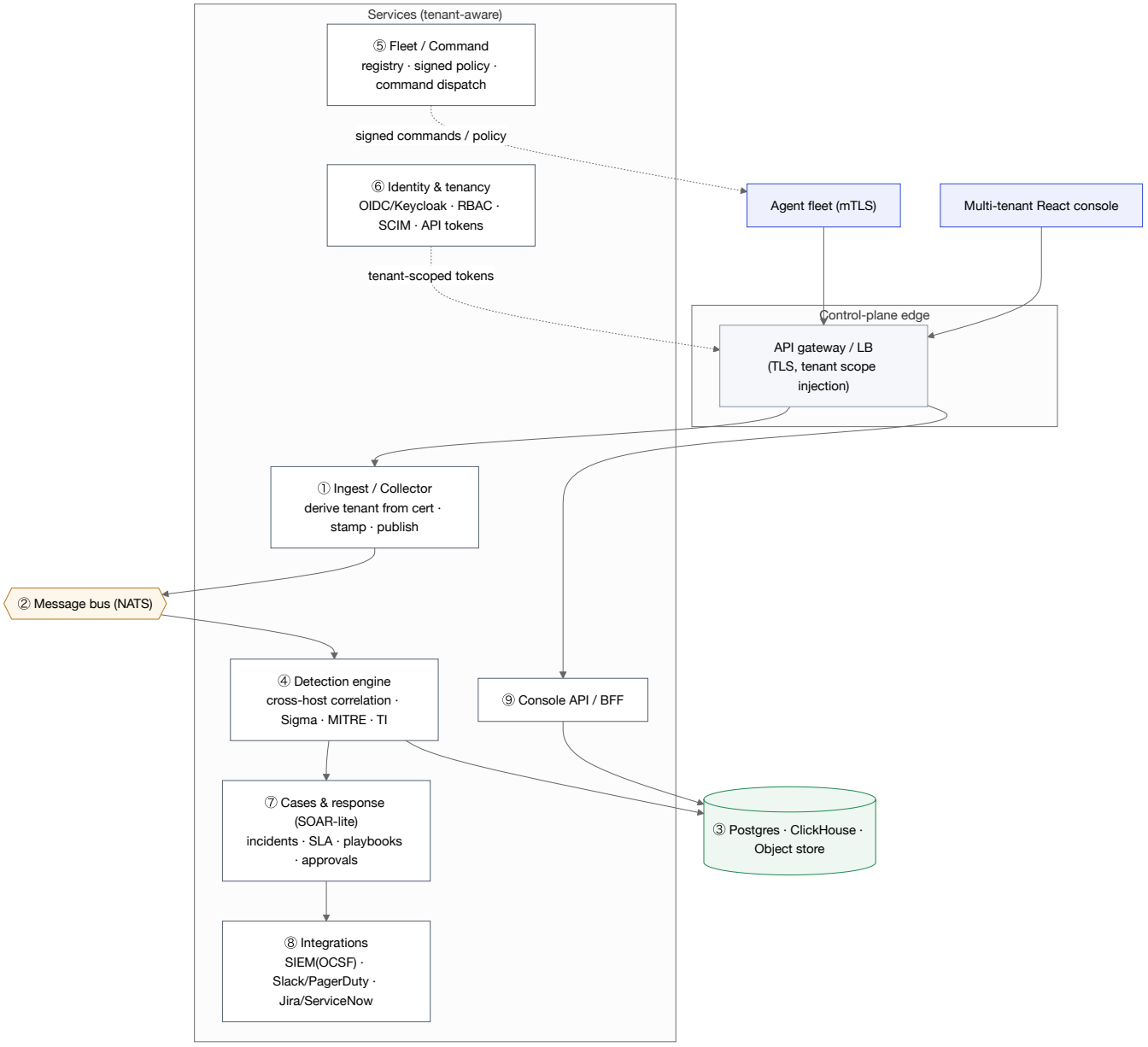


Figure 9.1 — Control-plane services. Numbering follows the architecture spec; the ingest collector is the central fan-in the single-host model lacked.

SERVICE	RESPONSIBILITY	STATUS
Ingest / Collector	Terminate agent mTLS; derive `tenant_id` + `agent_id` from the cert; validate, stamp tenant, publish to bus; per-tenant backpressure.	IMPLEMENTED
Identity & tenancy	OIDC via Keycloak 26; RBAC (MSOC-admin, tenant-analyst, read-only, cross-tenant responder); tenant-scoped tokens.	IMPLEMENTED
Fleet / Command	Agent registry; build + Ed25519-sign policy bundles; dispatch jail/thaw/kill-switch/preset/threshold with approval + audit.	IMPLEMENTED
Console API / BFF	Backend-for-frontend for the React console; every call tenant-scoped by the gateway.	IMPLEMENTED
Detection engine	Cross-host correlation within a tenant; Sigma→rule translation; ML/baseline anomaly.	PARTIAL / ROADMAP
Cases & response, Integrations	Case management + SLA; SIEM/SOAR/ticketing connectors.	ROADMAP

10 Agent ↔ control-plane wire contract

Five gRPC services (`ebpfsoc.v1`) define the protocol. All channels are gRPC over mTLS except the single bootstrap enrollment call. Identity is derived from the client certificate — **no request message carries `tenant_id`**, and that omission is load-bearing.

CHANNEL	SERVICE / RPC	DIR	SHAPE	PURPOSE
Enroll	`EnrollmentService.Enroll / Renew`	agent→CP	unary (bootstrap TLS)	One-time cert issuance; binds tenant+agent
Telemetry	`TelemetryService.StreamTelemetry`	agent→CP	bidi stream	Events/alerts/decisions; batched, deduped, resumable
Command	`CommandService.Commands`	CP→agent	bidi stream	mode/jail/thaw/thresholds/preset/kill-switch; signed, acked
Policy	`PolicyService.GetBundle`	agent→CP	unary	Signed policy bundle by etag; verify before apply
Heartbeat	`HeartbeatService.Heartbeat`	agent→CP	unary	Health, version, kernel, data-plane state, buffer depth

The command stream is *agent-initiated* (the agent dials out and holds the stream) so commands reach hosts behind customer NAT/firewalls without inbound connectivity.

10.1 Enrollment (mTLS bootstrap)

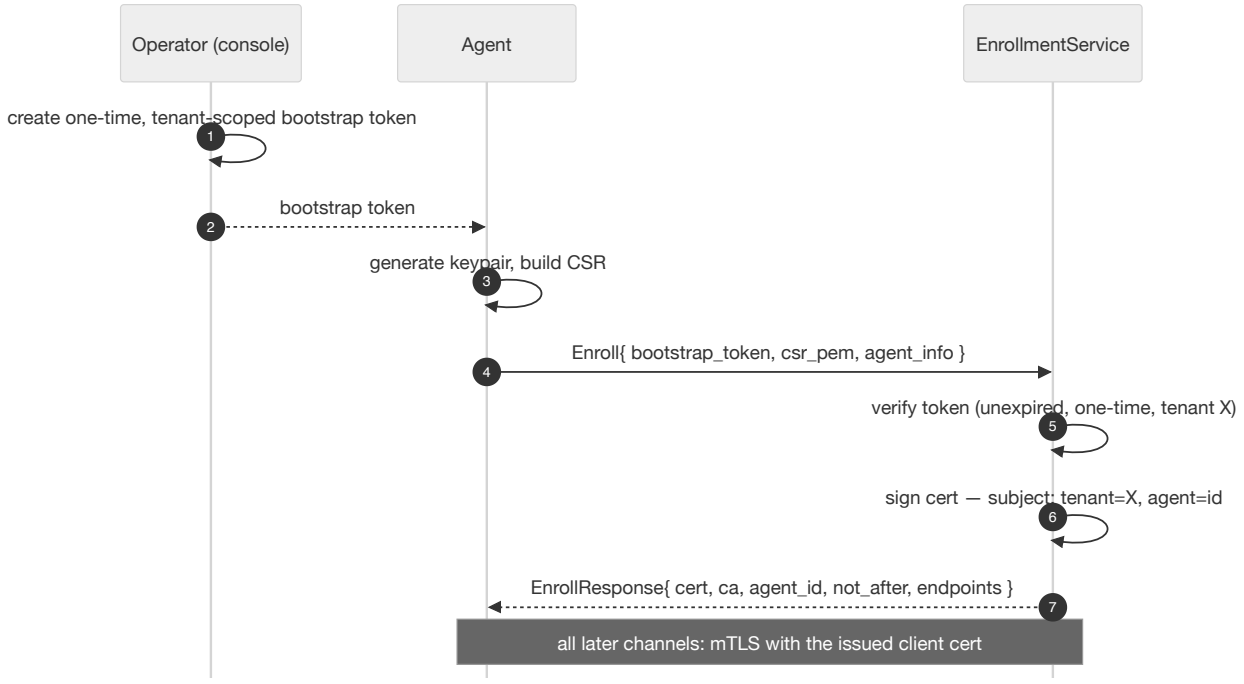


Figure 10.1 — Enrollment. Bootstrap tokens are short-lived, one-time and tenant-scoped; issued certs are short-lived and auto-rotated. An agent cannot change its tenant by re-issuing a CSR.

10.2 Telemetry uplink — at-least-once, idempotent, resumable

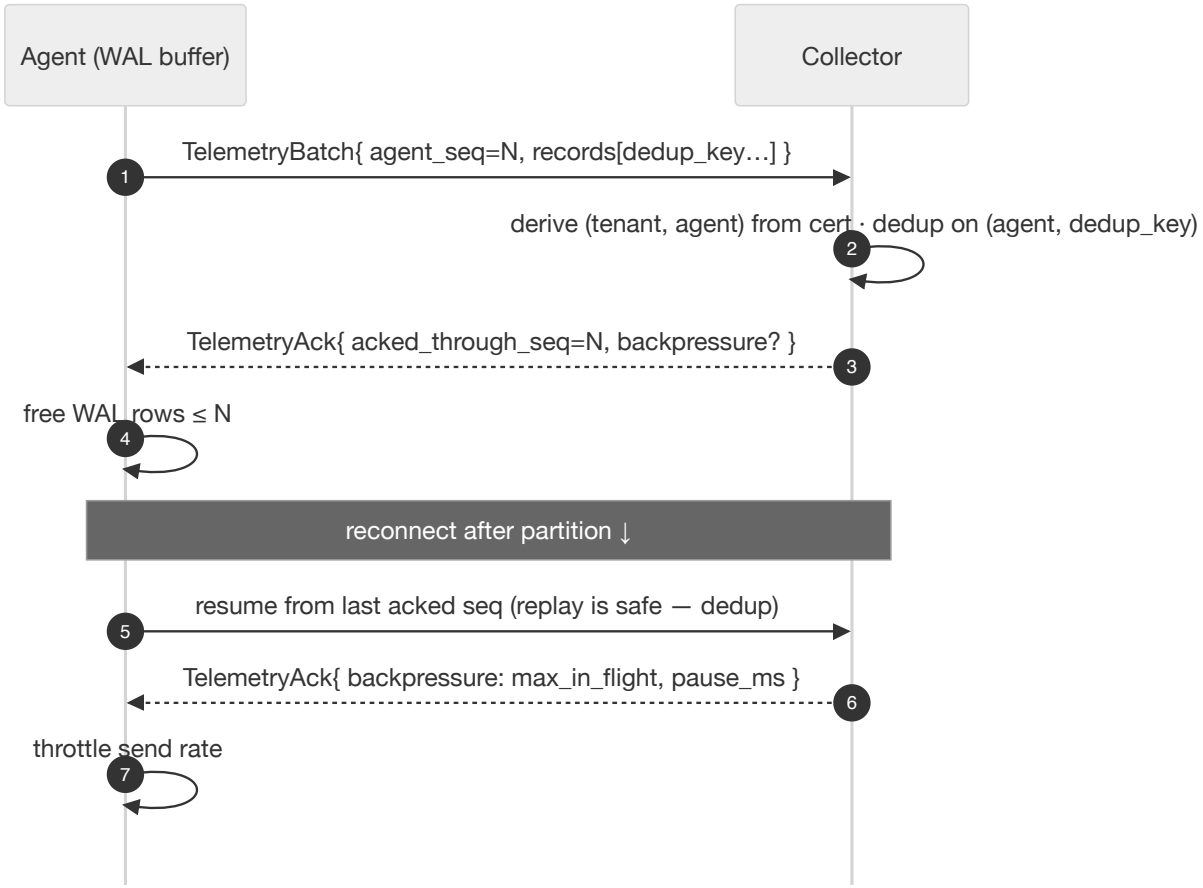


Figure 10.2 — Uplink semantics. Agent-assigned `dedup_key` makes replays harmless; cumulative `acked_through_seq` drives WAL truncation; per-agent backpressure protects other tenants.

10.3 Command channel — signed, guard-railed, acked

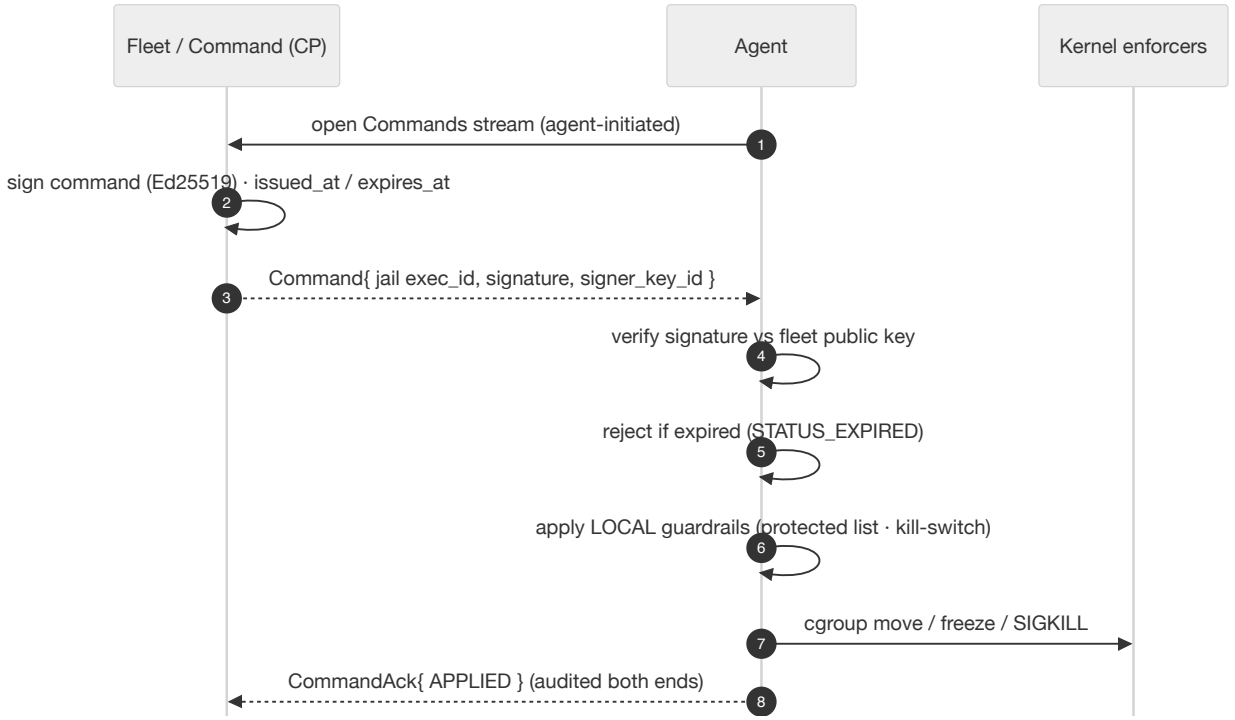
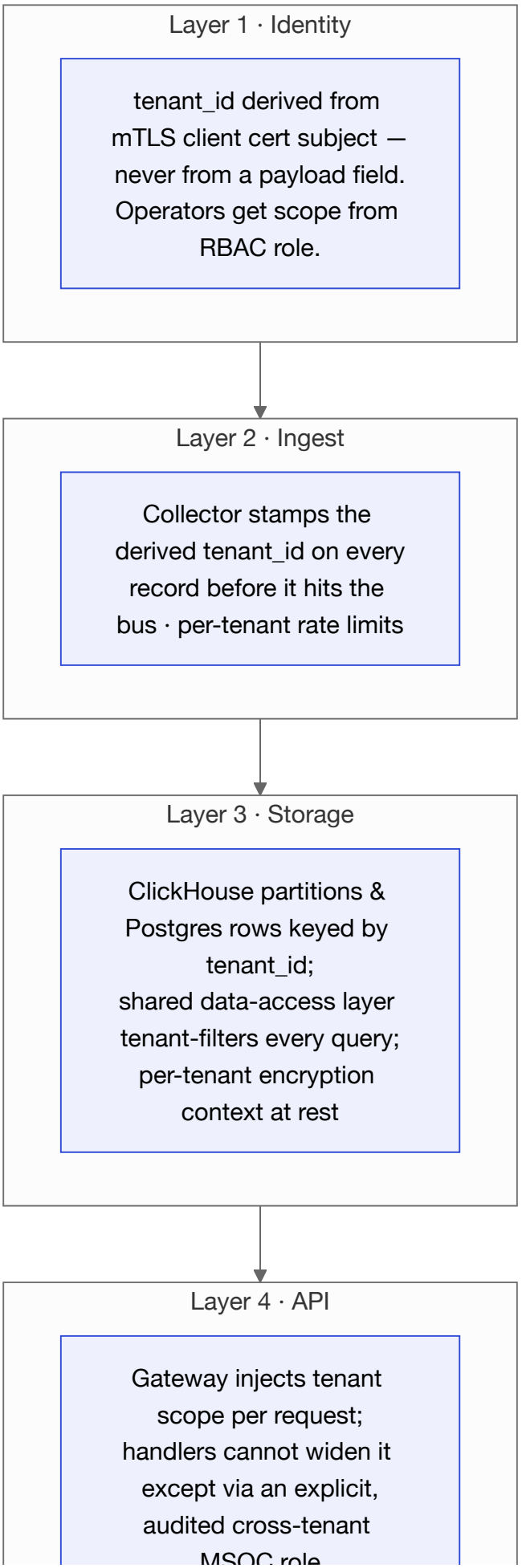


Figure 10.3 — "Signed-everything-downstream." Local guardrails override any command — a command cannot remove `sudo` / `sshd` from the protected set (the sudo-lockout trap).

11 Tenant isolation — the invariant

In an MSSP, one bug leaking another customer's data is existential. Isolation is therefore enforced at four independent layers, so no single defect breaches it.


Layer 1 · Identity
tenant_id derived from mTLS client cert subject — never from a payload field. Operators get scope from RBAC role.
Layer 2 · Ingest
Collector stamps the derived tenant_id on every record before it hits the bus · per-tenant rate limits
Layer 3 · Storage
ClickHouse partitions & Postgres rows keyed by tenant_id; shared data-access layer tenant-filters every query; per-tenant encryption context at rest
Layer 4 · API
Gateway injects tenant scope per request; handlers cannot widen it except via an explicit, audited cross-tenant MSOC role

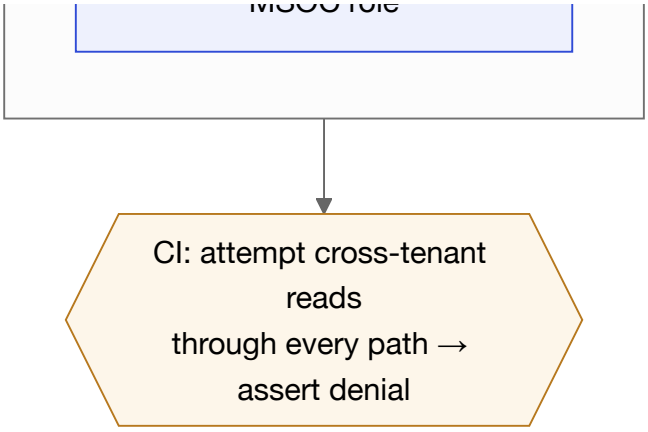


Figure 11.1 — Four-layer isolation. The invariant is verified by automated cross-tenant denial tests and an external pen-test before GA.

LOAD-BEARING OMISSION

No request message in the wire contract carries an authoritative `tenant_id`. A record that *did* could only ever be tamper-detection telemetry — never authorization. Tenant identity always comes from the verified certificate or the RBAC role.

12 Security model & threat coverage

12.1 Cryptographic posture

CONCERN	MECHANISM	WHERE
Agent ↔ CP transport & identity	mTLS; identity from cert subject	`internal/mtls`
Command / policy integrity	Ed25519 "signed-everything-downstream"; agents hold only the public key	`internal/signing`
Audit integrity	Hash-chained decisions; `/api/verify-chain`	`internal/store/decisions.go`
Operator identity (single-host)	bcrypt creds, HMAC-signed stateless cookie sessions, CSRF, login rate-limit	`internal/api/auth.go`
Operator identity (multi-tenant)	OIDC via Keycloak; RBAC; SCIM; API tokens	`internal/identity`, `internal/authz`

12.2 The autonomy contract (the moat)

Nothing in the protocol makes enforcement depend on the channel being up. If the control plane is unreachable, the agent keeps enforcing its last-applied signed policy and keeps buffering telemetry in the WAL, draining on reconnect via the resume semantics. Heartbeat/command/policy timeouts degrade to "operate on last known good" — *never* to "stop enforcing." This property is a required CI test (Phase-1 exit criterion: "still enforces when disconnected").

12.3 Local guardrails that override the cloud

- **Protected process/MAC lists** — `sudo`, `sshd`, the operator's management device, and infrastructure cannot be quarantined/severed, even by a signed command that tries to remove them from the set. This is the *sudo-lockout trap* defence.
- **Kill-switch** — a fleet-wide (and per-host) emergency halt of all enforcement.
- **Threshold calibration** — deploy defaults keep routine noise (sshd MOTD ~84) below quarantine so the operator never freezes their own access.

> **THREAT TRACEABILITY**
>
> The wire contract maps each decision to a threat ID: cert-derived tenant (CH-2), dedup+resume (CH-3), backpressure (CH-4), signed+acked commands (CH-5), local guardrails (EN-1/EN-4), signed policy (SC-4), one-time bootstrap + short-lived certs (SC-7/CH-6), and "no channel is a prerequisite for enforcement" (CH-7).

13 The operator console

A Vite multi-entry React SPA (TypeScript, Tailwind, Radix, D3, Zustand), built to a static bundle and embedded into the engine via `go:embed` in the single-host build, or served by nginx in the multi-tenant build. The information architecture follows the SOC lifecycle: detect → investigate → respond → report.

13.1 Surfaces

ROUTE	CONSOLE	PURPOSE
`/`	SOC dashboard	Executive summary, alert triage, MITRE coverage, IOCs, live event stream, correlation graph
`/choke`	Choke Gateway	State ladder, tracked processes, thresholds, manual override, kill-switch, policy workbench
`/devices`	Network Choke	Per-device (MAC) discovery, flows, jail/thaw, enforcement mode
`/fleet`	Fleet console	Drive N hosts as a unit (status, presets, thresholds, kill-switch)

13.2 Dual-mode containment surfaces (Command ⇄ Assurance)

The two Choke Gateways — the product's UVP — present the same live data through two lenses on one page, sharing a single `ContainmentCommand` header and ladder so the process plane and the network plane read as one product:

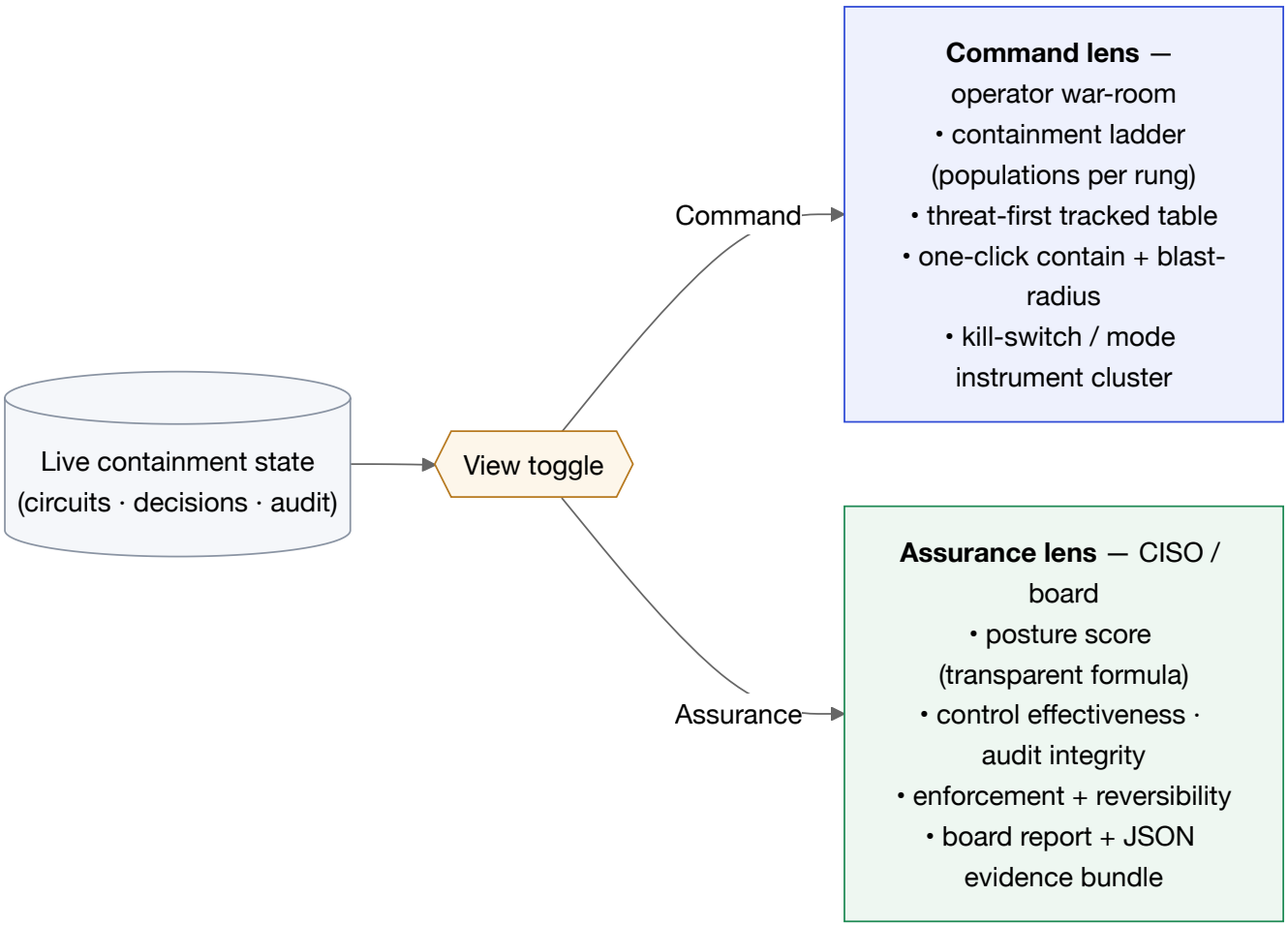


Figure 13.1 — One surface, two audiences. The Command lens serves the operator during an incident; the Assurance lens serves the buyer/board during review.

Process and alert drill-downs carry **dual narratives** — a plain-English summary (severity, behaviour, MITRE technique, recommended action) beside the technical detail — so the same evidence reads for both an analyst and a non-technical stakeholder. Real-time updates arrive over Server-Sent Events; every alert, event and decision fans out to all connected consoles.

14 End-to-end event lifecycle

From an attacker's syscall to a kernel containment action and a live console update — typically in well under a second.

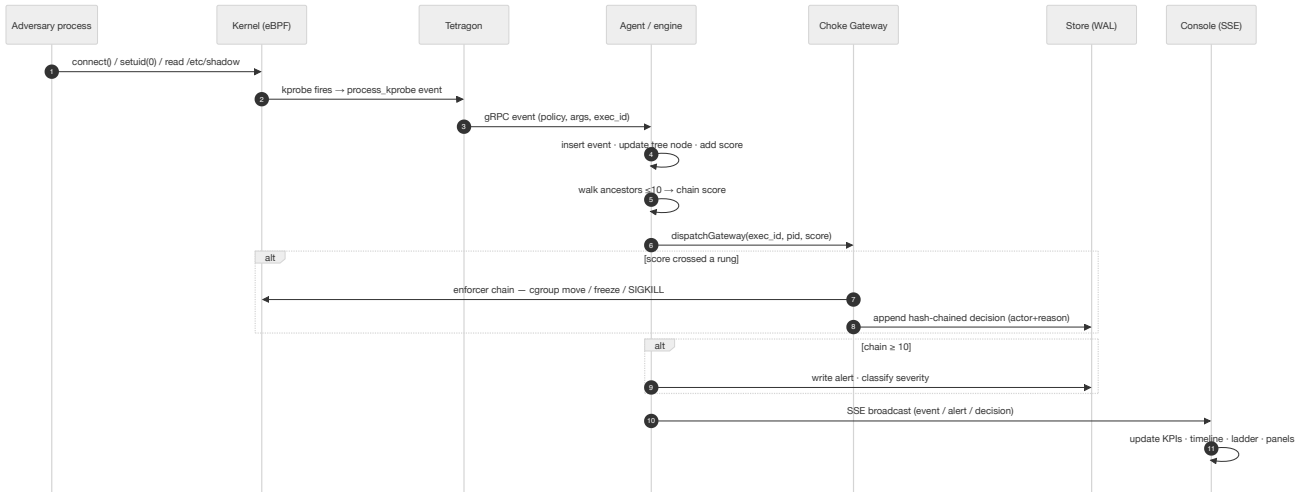


Figure 14.1 — The full path. Note the gateway is called on every event, so a process can be throttled before it ever raises an alert.

15 Deployment topologies & modes

15.1 Enforcement / operating modes

MODE	FLAG	BEHAVIOUR
Detect-only	`enforce: false`	Decisions audited, kernel untouched. The safe default.
Enforcing (process)	`-enforce`	Real cgroup throttle/freeze + SIGKILL on this host; flippable at runtime via `/api/choke/mode`.
Enforcing (device)	`-devchoke-obj/-iface`	Inline-bridge tc data plane; independent of `-enforce`.
Dry-run	`-dry-run`	Records decisions, executes nothing — shadow a new policy safely.
Fake	`-fake`	Synthesizes events, no Tetragon — UI iteration / unit tests only.

15.2 Topologies

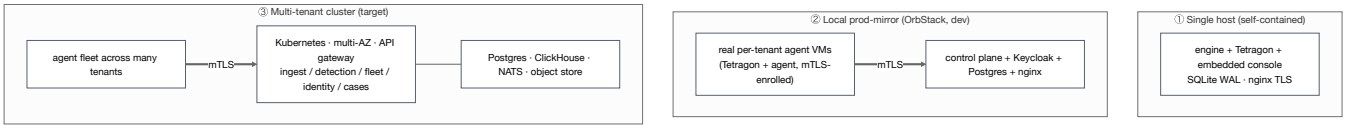


Figure 15.1 — Three topologies over the same core: a single self-contained host, a local prod-mirror (used for development), and the target multi-tenant cluster.

The reference environment runs a local prod-mirror: a control plane (Keycloak + Postgres + nginx) with real per-tenant Alpine agent VMs running Tetragon and the agent binary, mTLS-enrolled and streaming genuine kernel telemetry — the same code paths as the target cluster, at laptop scale.

16 Technology stack

LAYER	TECHNOLOGY	WHY
Kernel telemetry	Tetragon v1.6.1 (CO-RE eBPF) + Go client	Pre-written CO-RE programs, declarative TracingPolicy, stable exec_id, gRPC stream
Kernel enforcement	Hand-written eBPF C (clang) + `cilium/ebpf` v0.21 (pure-Go loader)	Per-PID token buckets & MAC-keyed shaping Tetragon can't express; keeps binary static
Non-BPF enforcement	cgroup v2 (cpu.max, freezer), seccomp, SIGKILL	CPU throttle, full freeze, syscall denial — the rungs below sever
Engine / agent / CP	Go 1.25 · 5 binaries	Single static binary per role, trivial cross-compile
Wire protocol	gRPC 1.80 + Protobuf 1.36 · `ebpfsoc.v1`	Five typed services with streaming telemetry
Transport auth / integrity	mTLS · Ed25519 signing	Unspoofable agent identity; forge-proof commands/policy
Identity / SSO	Keycloak 26 + OIDC · RBAC	Multi-tenant realms, token lifecycle, MSSP model
Storage	SQLite WAL (edge) · Postgres 16 · ClickHouse 24.8 · NATS · object store	Pure-Go edge buffer; relational SoR; columnar analytics; fan-out bus
Console	React 18 · TS 5.7 · Vite 6 · Tailwind · Radix · D3 · Zustand	Static bundle embeddable via go:embed; small, accessible
Observability	OpenTelemetry SDK + OTLP/HTTP	Vendor-neutral metrics export
Packaging	Docker/compose · systemd · OrbStack · Terraform	OSS compose stack; bare-metal units; local mirror; IaC clusters
Testing	Vitest · Playwright 1.49 e2e · Go testing	Unit both sides + real-browser E2E against the deployed stack

17 Limitations & roadmap

17.1 Current limitations

- **Device choke is operator-driven.** A forwarding node has no per-device Tetragon telemetry, so there is no automatic score path for the MAC gateway yet — manual jail/thaw with an audit trail.
- **One L2 hop for device choke.** MAC keying assumes the gateway is the device's L2 neighbour; devices behind a downstream router collapse to that router's MAC.
- **Device IP/hostname enrichment.** The central `DeviceSummary` does not yet serialize `last_ip`; the console uses a LAN-IP heuristic until the proto is extended.
- **Fixed scoring rules.** No baseline learning yet; tuning is manual in `scorer.go` and via thresholds.

17.2 Roadmap themes

- Cross-host correlation and Sigma→rule detection-as-code; ML/baseline anomaly models.
- Case management + SLA, and outbound integrations (SIEM via OCSF, SOAR, ticketing).
- Admin/governance surfaces: RBAC management, tenant administration, an admin/user action audit log, data-retention/residency controls.
- Device risk scoring so the MAC gateway gains an automatic score path.

> **THE THROUGH-LINE**
> Every roadmap item is additive to a core that already does the hard, differentiating thing: **graduated, reversible, cryptographically-audited enforcement at the kernel and network layer** — the capability the rest of the market largely leaves at detect-and-alert.

A Appendices

A.1 HTTP API surface (single-host engine)

CORE / SOC

`GET /api/{whoami,events,alerts,process/<exec_id>,stream,policies,attacks,honeypots,policy-stats,origin,version,system-health,decisions,verify-chain}` · `POST /api/{login,run-attack}` · `GET /api/logout`. All non-public paths require the `soc_session` cookie; unsafe writes require the CSRF header.

PROCESS CHOKE (`/CHOKE`)

`GET /api/choke/{state,circuits,buckets,thresholds,policies,cgroups,processes,process/<id>,proc/<pid>}` · `POST /api/choke/{manual,bulk-manual,jail,forget,thaw,annotate,kill-switch,preset,mode,forensic-snapshot}` · `PUT /api/choke/thresholds` · `POST /api/choke/policy/preview`.

DEVICE CHOKE (`/DEVICES`)

`GET /api/choke/{devices,device-state,device-flows}` · `POST /api/choke/{device-jail,device-thaw,device-mode,device-kill-switch}`.

FLEET (`/FLEET`)

`GET /api/fleet/{hosts,state,cgroups,decisions,alerts,devices}` · `POST /api/fleet/{preset,kill-switch,thaw,device-jail}` · `PUT /api/fleet/thresholds`.

A.2 Supporting BPF maps

MAP	TYPE / SIZE	PURPOSE
`choke_pids`	HASH · 65,536	`u32 pid` → 24-byte bucket (flags + token bucket). Read by programs 1–4.
`choke_devs`	HASH · 4,096	`dev_key{mac[6]+pad}` → 24-byte bucket. Read by programs 5–6.
`choke_devs_seen`	LRU_HASH · 4,096	Passive discovery — last-seen, packet count, sampled source IPv4.
`choke_flows`	LRU_HASH · 8,192	Per-device destination flows (mac, daddr, dport, proto → pkts/bytes/last-seen).

A.3 Glossary

`exec_id`	Tetragon's stable per-execution process identifier; survives PID reuse. The unit of process control.
Chain score	Sum of per-event scores along the ancestor walk (≤10 hops) — the risk number driving alerting and choke.
Rung	One of the five choke states: pristine · throttled · tarpit · quarantined · severed.
Choke	The graduated, reversible-where-it-matters enforcement action applied to a process or device.
Autonomy contract	The guarantee that kernel enforcement never depends on the control plane being reachable.
Dedup key	Agent-assigned, resend-stable telemetry id enabling at-least-once + idempotent uplink.
Honeypot signal	A read of a seeded decoy credential file — a high-confidence indicator with no benign explanation.

Generated from the eBPF-SOC source tree and the `/docs` corpus (31 documents). Diagrams rendered with Mermaid. For deeper detail on any subsystem, the cited source paths (`engine/internal/…`) and companion docs (`docs/architecture/` , `docs/plan/`) are the authoritative references.